

通过前面的章节，我们已经了解了一些基本的数据结构。根据其实现方式，这些数据结构大致可以分为两种类型：基于数组的实现与基于链表的实现。正如我们已经看到的，就其效率而言，二者各有长短。具体来说，前一实现方式允许我们通过下标或秩，在常数的时间内找到目标对象；然而，一旦需要对这类结构进行修改，那么无论是插入还是删除，都需要耗费线性的时间。反过来，后一实现方式允许我们借助引用或位置对象，在常数的时间内插入或删除元素；但是为了找出居于特定次序的元素，我们却不得不花费线性的时间，对整个结构进行遍历查找。能否将这两类结构的优点结合起来，并回避其不足呢？本章所讨论的树结构，将正面回答这一问题。

在此前介绍的这些结构中，元素之间都存在一个自然的线性次序，故它们都属于所谓的线性结构（**linear structure**）。树则不然，其中的元素之间并不存在天然的直接后继或直接前驱关系。不过，正如我们马上就要看到的，只要附加某种约束（比如遍历），也可以在树中的元素之间确定某种线性次序，因此树属于半线性结构（**semi-linear structure**）。

无论如何，随着从线性结构转入树结构，我们的思维方式也将有个飞跃；相应地，算法设计的策略与模式也会因此有所变化，许多基本的算法也将得以更加高效地实现。以第7章和第8章将要介绍的平衡二叉搜索树为例，若其中包含 n 个元素，则每次查找、更新、插入或删除操作都可在 $O(\log n)$ 时间内完成——相对于线性结构，几乎提高了一个线性因子（习题[1-9]）。

树结构有着不计其数的变种，在算法理论以及实际应用中，它们都扮演着最为关键的角色。之所以如此，是因得益于其独特而又普适的逻辑结构。树是一种分层结构，而层次化这一特征几乎蕴含于所有事物及其联系当中，成为其本质属性之一。从文件系统、互联网域名系统和数据库系统，一直到地球生态系统乃至人类社会系统，层次化特征以及层次结构均无所不在。

有趣的是，作为树的特例，二叉树实际上并不失其一般性。本章将指出，无论就逻辑结构或算法功能而言，任何有根有序的多叉树，都可等价地转化并实现为二叉树。因此，本章讲解的重点也将放在二叉树上。我们将以通讯编码算法的实现这一应用实例作为线索贯穿全章。

§ 5.1 二叉树及其表示

5.1.1 树

■ 有根树

从图论的角度看，树等价于连通无环图。因此与一般的图相同，树也由一组顶点（**vertex**）以及联接与其间的若干条边（**edge**）组成。在计算机科学中，往往还会在此基础上，再指定某一特定顶点，并称之为根（**root**）。在指定根节点之后，我们也称之为有根树（**rooted tree**）。此时，从程序实现的角度，我们也更多地将顶点称作节点（**node**）。

■ 深度与层次

由树的连通性，每一节点与根之间都有一条路径相联；而根据树的无环性，由根通往每个节点的路径必然唯一。因此如图5.1所示，沿每个节点 v 到根 r 的唯一通路所经过边的数目，称作 v 的深度（depth），记作 $\text{depth}(v)$ 。依据深度排序，可对所有节点做分层归类。特别地，约定根节点的深度 $\text{depth}(r) = 0$ ，故属于第0层。

■ 祖先、后代与子树

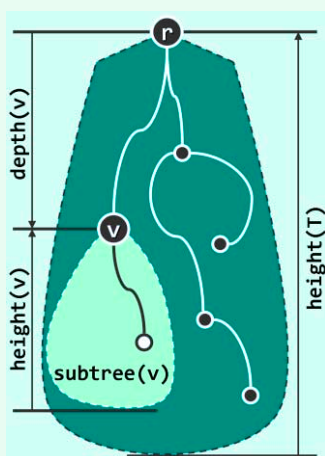


图5.1 有根树的逻辑结构

任一节点 v 在通往树根沿途所经过的每个节点都是其祖先（ancestor）， v 是它们的后代（descendant）。特别地， v 的祖先/后代包括其本身，而 v 本身以外的祖先/后代称作真祖先（proper ancestor）/真后代（proper descendant）。

节点 v 历代祖先的层次，自下而上以1为单位逐层递减；在每一层次上， v 的祖先至多一个。特别地，若节点 u 是 v 的祖先且恰好比 v 高出一层，则称 u 是 v 的父亲（parent）， v 是 u 的孩子（child）。

v 的孩子总数，称作其度数或度（degree），记作 $\text{deg}(v)$ 。无孩子的节点称作叶节点（leaf），包括根在内的其余节点皆为内部节点（internal node）。

v 所有的后代及其之间的联边称作子树（subtree），记作 $\text{subtree}(v)$ 。在不致歧义时，我们往往不再严格区分节点（ v ）及以之为根的子树（ $\text{subtree}(v)$ ）。

■ 高度

树 T 中所有节点深度的最大值称作该树的高度（height），记作 $\text{height}(T)$ 。

不难理解，树的高度总是由其中某一叶节点的深度确定的。特别地，本书约定，仅含单个节点的树高度为0，空树高度为-1。

推而广之，任一节点 v 所对应子树 $\text{subtree}(v)$ 的高度，亦称作该节点的高度，记作 $\text{height}(v)$ 。特别地，全树的高度亦即其根节点 r 的高度， $\text{height}(T) = \text{height}(r)$ 。

5.1.2 二叉树

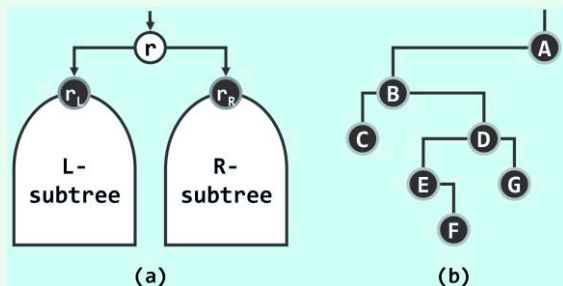


图5.2 二叉树：(a)逻辑结构；(b)实例

如图5.2所示，二叉树（binary tree）中每个节点的度数均不超过2。

因此在二叉树中，同一父节点的孩子都可以左、右相互区分——此时，亦称作有序二叉树（ordered binary tree）。本书所提到的二叉树，默认地都是有序的。

特别地，不含一度节点的二叉树称作真二叉树（proper binary tree）（习题[5-2]）。

5.1.3 多叉树

一般地，树中各节点的孩子数目并不确定。每个节点的孩子均不超过 k 个的有根树，称作 k 叉树（ k -ary tree）。本节将就此类树结构的表示与实现方法做一简要介绍。

父节点

由如图5.3(a)实例不难看出，在多叉树中，根节点以外的任一节点有且仅有一个父节点。

因此可如图5.3(b)所示，将各节点组织为向量或列表，其中每个元素除保存节点本身的信息（node）外，还需要保存父节点（parent）的秩或位置。可为树根指定一个虚构的父节点-1或NULL，以便统一判断。

如此，所有向量或列表所占的空间总量为 $O(n)$ ，线性正比于节点总数 n 。时间方面，仅需常数时间，即可确定任一节点的父节点；但反过来，孩子节点的查找却不得不花费 $O(n)$ 时间访遍所有节点。

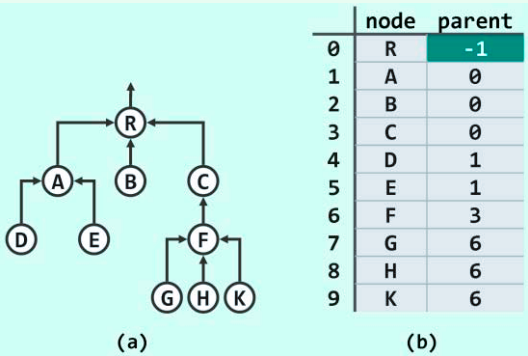


图5.3 多叉树的“父节点”表示法

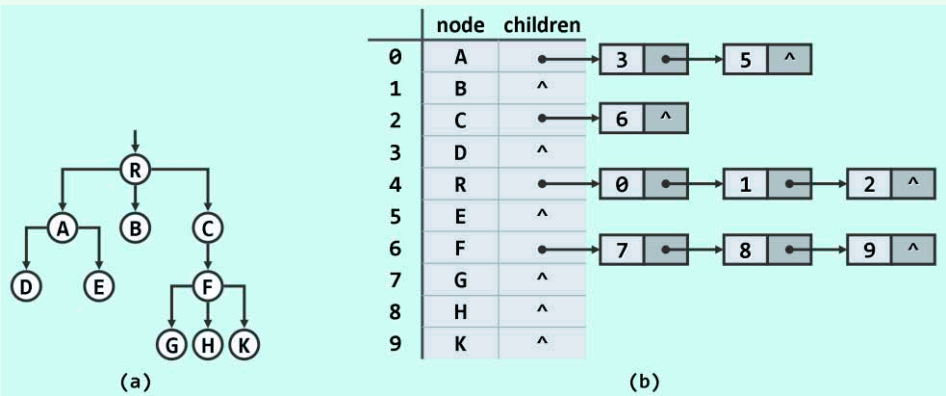


图5.4 多叉树的“孩子节点”表示法

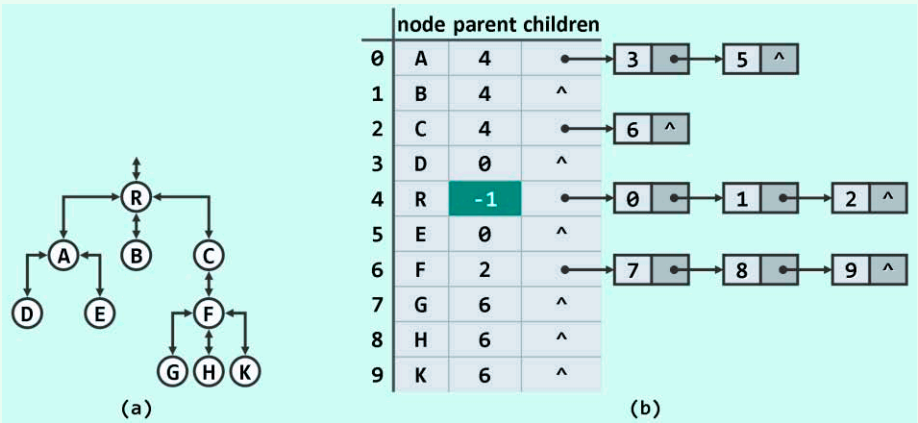


图5.5 多叉树的“父节点 + 孩子节点”表示法

■ 孩子节点

若注重孩子节点的快速定位,可如图5.4所示,令各节点将其所有的孩子组织为一个向量或列表。如此,对于拥有 r 个孩子的节点,可在 $O(r + 1)$ 时间内列举出其所有的孩子。

■ 父节点 + 孩子节点

以上父节点表示法和孩子节点表示法各有所长,但也各有所短。为综合二者的优势,消除缺点,可如图5.5所示令各节点既记录父节点,同时也维护一个序列以保存所有孩子。

尽管如此可以高效地兼顾对父节点和孩子的定位,但在节点插入与删除操作频繁的场所,为动态地维护和更新树的拓扑结构,不得不反复地遍历和调整一些节点所对应的孩子序列。然而,向量和列表等线性结构的此类操作都需耗费大量时间,势必影响到整体的效率。

■ 有序多叉树 = 二叉树

解决上述难题的方法之一,就是采用支持高效动态调整的二叉树结构。为此,必须首先建立起从多叉树到二叉树的某种转换关系,并使得在此转换的意义下,任一多叉树都等价于某棵二叉树。当然,为了保证作为多叉树特例的二叉树有足够的力量表示任何一棵多叉树,我们只需给多叉树增加一项约束条件——同一节点的所有孩子之间必须具有某一线性次序。

仿照有序二叉树的定义,凡符合这一条件的多叉树也称作有序树(ordered tree)。幸运的是,这一附加条件在实际应用问题中往往自然满足。以互联网域名系统所对应的多叉树为例,其中同一域名下的分支通常即按照字典序排列。

■ 长子 + 兄弟

由图5.6(a)的实例可见,有序多叉树中任一非叶节点都有唯一的“长子”,而且从该“长子”出发,可按照预先约定或指定的次序遍历所有孩子节点。因此可如图(b)所示,为每个节点设置两个指针,分别指向其“长子”和下一“兄弟”。

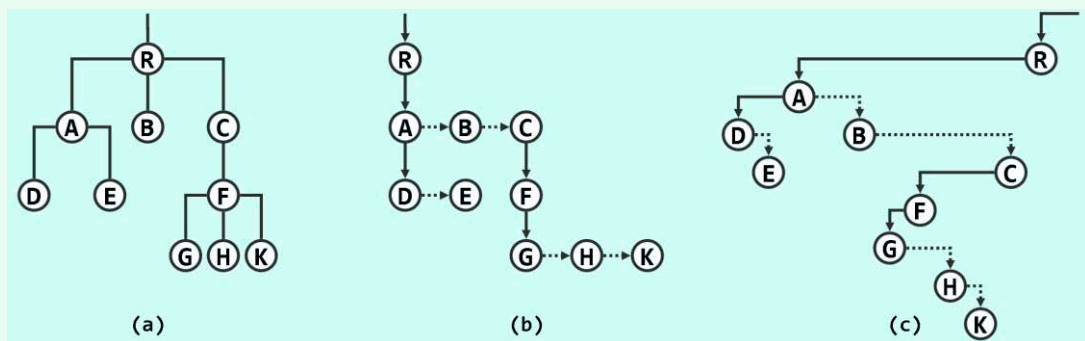


图5.6 多叉树的“长子 + 兄弟”表示法 ((b)中长子和兄弟指针,分别以垂直实线和水平虚线示意)

现在,若将这两个指针分别与二叉树节点的左、右孩子指针统一对应起来,则可进一步地将原有序多叉树转换为如图(c)所示的常规二叉树。

在这里,一个饶有趣味的现象出现了:尽管二叉树只是多叉树的一个子集,但其对应用问题的描述与刻画能力绝不低于后者。实际上以下我们还将进一步发现,即便是就计算效率而言,二叉树也并不逊色于一般意义上的树。反过来,得益于其定义的简洁性以及结构的规范性,二叉树所支撑的算法往往可以更好地得到描述,更加简捷地得到实现。二叉树的身影几乎出现在所有的应用领域当中,这也是一个重要的原因。

§ 5.2 编码树

本章将以通讯编码算法的实现作为二叉树的应用实例。通讯理论中的一个基本问题是，如何在尽可能低的成本下，以尽可能高的速度，尽可能忠实地实现信息在空间和时间上的复制与转移。在现代通讯技术中，无论采用电、磁、光或其它任何形式，在信道上传递的信息大多以二进制比特的形式表示和存在，而每一个具体的编码方案都对应于一棵二叉编码树。

5.2.1 二进制编码

在加载到信道上之前，信息被转换为二进制形式的过程称作编码（encoding）；反之，经信道抵达目标后再由二进制编码恢复原始信息的过程称作解码（decoding）。

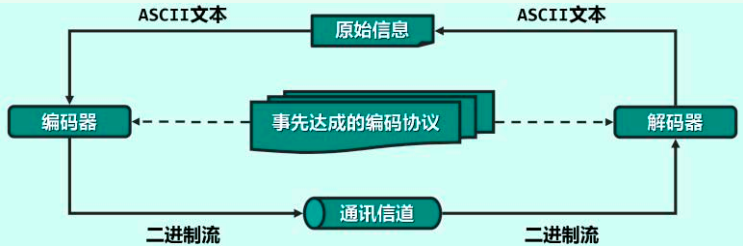


图5.7 完整的通讯过程由预处理、编码和解码阶段组成

如图5.7所示，编码和解码的任务分别由发送方和接收方分别独立完成，故在开始通讯之前，双方应已经以某种形式，就编码规则达成过共同的约定或协议。

■ 生成编码表

原始信息的基本组成单位称作字符，它们都来自于某一特定的有限集合 Σ ，也称作字符集（alphabet）。而以二进制形式承载的信息，都可表示为来自编码表 $\Gamma = \{ 0, 1 \}^*$ 的某一特定二进制串。从这个角度理解，每一编码表都是从字符集 Σ 到编码表 Γ 的一个单射，编码就是对信息文本中各字符逐个实施这一映射的过程，而解码则是逆向映射的过程。

表5.1 $\Sigma = \{ 'A', 'E', 'G', 'M', 'S' \}$ 的一份二进制编码表

字符	A	E	G	M	S
编码	00	01	10	110	111

表5.1即为二进制编码表的实例。编码表一旦制定，信息的发送方与接收方之间也就建立起了一个约定与默契，从而使得独立的编码与解码成为可能。

■ 二进制编码

现在，所谓编码就是对于任意给定的文本，通过查阅编码表逐一将其中的字符转译为二进制编码，这些编码依次串接起来即得到了全文的编码。比如若待编码文本为"MESSAGE"，则根据由表5.1确定的编码方案，对应的二进制编码串应为"110⁰¹111¹¹¹00¹⁰01"^①。

表5.2 二进制解码过程

二进制编码	当前匹配字符	解出原文
11001111111001001	M	M
011111111001001	E	ME
111111001001	S	MES
111001001	S	MESS
001001	A	MESSA
1001	G	MESSAG
01	E	MESSAGE

① 这里对各比特位做了适当的上下移位，以便读者区分各字符编码串的范围；在实际编码中，它们并无“高度”的区别